



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

Aplicaciones para Comunicaciones en Red

TEMA:

Práctica: Cliente Wget Con hilos

ALUMNO:

Erreguin Franco Yair Alejandro

Boleta: 2022630121

PROFESOR:

Moreno Cervantes Axel Ernesto

24 de junio de 2026

Índice

1. Introducción	2
1.1. Web Crawling y Wget	2
1.2. Protocolo de Transferencia de Hipertexto (HTTP)	2
1.2.1. Formato Estructural de la Solicitud GET	3
1.2.2. Formato Estructural de la Respuesta del Servidor	3
1.3. Análisis de Contenido HTML con Jsoup	3
1.4. Concurrencia y Thread Pools	4
2. Desarrollo	5
2.1. Método main	5
2.2. Método descarga	6
2.3. Método descargarElementoConHilo	8
3. Ejecución	10
3.1. Prueba con Profundidad 0 (Solo página base y sus elementos)	10
3.2. Prueba con Profundidad 2 (Descarga acotada)	10
4. Conclusiones	13
5. Referencias	14

1. Introducción

El internet constituye un ecosistema digital de documentos y recursos interconectados a través de hipervínculos. La capacidad de navegar y recuperar esta información de manera automática es fundamental para motores de búsqueda, análisis de datos y la preservación de contenido digital. Herramientas como GNU Wget han establecido un estándar para la descarga recursiva de sitios web, permitiendo a los usuarios crear “espejos” locales de contenido en línea.

El objetivo de esta práctica es diseñar y desarrollar en Java una aplicación de consola que emule la funcionalidad de un web crawler. A partir de una URL proporcionada por el usuario, el programa deberá ser capaz de descargar el recurso principal, analizar su contenido para identificar todas las dependencias como hojas de estilo (CSS), scripts (JavaScript), imágenes y otros hipervínculos. Posteriormente, descargar estos recursos de forma recursiva hasta obtener una copia local y navegable del sitio.

Para lograr un rendimiento óptimo frente al gran número de peticiones de red requeridas, la solución implementa un modelo de concurrencia basado en un pool de hilos. Este gestionará un conjunto de hilos de trabajo que procesarán las descargas en paralelo. La extracción de enlaces de los documentos HTML se realizará con la ayuda de la biblioteca Jsoup, especializada en el análisis de HTML, mientras que la comunicación a bajo nivel con los servidores web se manejará a través de la clase `URLConnection` de Java, que implementa el protocolo HTTP.

1.1. Web Crawling y Wget

El web crawling es el proceso mediante el cual un programa automatizado navega por el internet de forma metódica[3]. Su función principal es indexar el contenido de los sitios web. Comienza con una lista de URLs a visitar y, en cada página, identifica todos los hipervínculos para añadirlos a su lista de URLs por visitar.

Wget es una popular herramienta de línea de comandos de software libre cuya función principal es la descarga de contenidos desde servidores web. Su nombre deriva de *World Wide Web* y *get*. Entre sus características más destacadas se encuentra la descarga recursiva de sitios web completos, la conversión de enlaces para su visualización local y la capacidad de reanudar descargas interrumpidas. Nuestra aplicación se inspira en esta funcionalidad de descarga recursiva.

1.2. Protocolo de Transferencia de Hipertexto (HTTP)

El Protocolo de Transferencia de Hipertexto (HTTP) es la base de la comunicación de datos en la Web[4]. Funciona como un protocolo de petición-respuesta entre un cliente y un servidor. El cliente (nuestra aplicación) envía una petición HTTP a un servidor para solicitar un recurso, identificado por una URL.

Una petición HTTP consiste en:

- **Método:** Indica la acción a realizar. Para nuestra herramienta, el método principal será `GET`, que solicita una representación de un recurso específico.

- **Cabeceras (Headers):** Contienen metadatos sobre la petición. Es común incluir una cabecera `User-Agent` para que el servidor identifique al cliente. Nuestra aplicación simulará ser un navegador Mozilla para evitar ser bloqueada.
- **Cuerpo (Body):** Para las peticiones GET, el cuerpo está vacío.

1.2.1. Formato Estructural de la Solicitud GET

Para comprender la transferencia a nivel de bytes en el flujo de transporte, la solicitud GET generada por el socket debe seguir la estructura estricta dictada por el protocolo, donde las cabeceras terminan obligatoriamente con una línea en blanco compuesta por un doble retorno de carro y salto de línea (`\r\n\r\n`):

```
GET /axel/aplicaciones/sockets/java/ HTTP/1.1
Host: 148.204.58.221
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml
[Línea en blanco explícita \r\n]
```

1.2.2. Formato Estructural de la Respuesta del Servidor

Tras procesar la petición, el servidor devuelve un flujo de datos que comienza con una línea de estado, un conjunto de metadatos en las cabeceras, una línea en blanco delimitadora, y finalmente el payload correspondiente a los bytes del recurso:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jun 2026 23:11:00 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: text/html; charset=UTF-8
Content-Length: 1024
Connection: close
[Línea en blanco explícita \r\n]
<!DOCTYPE html><html><head>...[Bytes del cuerpo]...
```

El cliente lee esta estructura para interpretar el código de estado (por ejemplo, 200 OK si tuvo éxito) antes de descargar el recurso solicitado.

1.3. Análisis de Contenido HTML con Jsoup

Una vez que se descarga un archivo HTML, es necesario analizar su estructura para encontrar enlaces a otros recursos. Realizar este análisis manualmente con expresiones regulares es complejo y propenso a errores.

Jsoup es una biblioteca de código abierto para Java diseñada para trabajar con HTML[2]. Proporciona una API muy conveniente para extraer y manipular datos, utilizando los mejores métodos de DOM, CSS y jQuery. Con Jsoup, podemos cargar un documento HTML desde un archivo o una cadena y usar selectores de CSS para encontrar elementos. Por ejemplo, para encontrar todas las imágenes y enlaces, podemos usar el selector `[src]`, `[href]`.

1.4. Concurrency y Thread Pools

Descargar un sitio web completo implica realizar cientos o miles de peticiones de red. Si estas peticiones se realizan de forma secuencial, el proceso sería extremadamente lento, ya que cada descarga tendría que esperar a que la anterior termine. La concurrencia permite que múltiples tareas se ejecuten de manera superpuesta. En Java, esto se logra comúnmente con hilos (*Threads*). Sin embargo, crear un nuevo hilo para cada descarga es ineficiente y puede agotar los recursos del sistema.

Un Thread Pool (piscina de hilos) es un patrón de diseño que consiste en un conjunto de hilos de trabajo gestionados. En lugar de crear un hilo por cada tarea, las tareas se envían a una cola y los hilos del pool las toman y ejecutan. Esto limita el número de hilos activos simultáneamente y reutiliza los hilos existentes, mejorando significativamente el rendimiento y la escalabilidad. La clase **ExecutorService** configurada mediante un número estático de hilos es la implementación principal utilizada en esta práctica[1].

2. Desarrollo

Toda la lógica del sistema se encuentra encapsulada dentro de la clase `Prueba.java`. Esta clase se encarga de recibir los parámetros del usuario, administrar las estructuras globales, gestionar de forma aislada las descargas de elementos de la capa de presentación mediante un pool cerrado de hilos y truncar la exploración de enlaces a través de una variable de cuenta regresiva explícita.

```
1 import java.util.Scanner;
2 import java.util.HashSet;
3 import java.util.Set;
4 import java.util.concurrent.ExecutorService;
5 import java.util.concurrent.Executors;
6 import org.jsoup.Jsoup;
7 import org.jsoup.nodes.Document;
8 import org.jsoup.nodes.Element;
9 import org.jsoup.select.Elements;
10 import java.nio.file.Files;
11 import java.nio.file.Path;
12 import java.nio.file.Paths;
13 import java.io.*;
14 import java.net.*;
15
16 public class Prueba {
17     private static Set<String> visited = new HashSet<>();
18     private static ExecutorService poolHilos = Executors.
        newFixedThreadPool(5);
```

Listing 1: Estructura base de la clase `Prueba.java`

2.1. Método `main`

Actúa como el punto de entrada principal del programa. Solicita de forma explícita al usuario la URL semilla y el nivel entero de profundidad. Una vez leídos los datos, parsea la cadena mediante la clase `URI` para aislar el nombre del host remoto y crear la carpeta raíz en disco. Posteriormente, delega la carga inicial a la función `descarga` inyectando el nivel entero puro, y finalmente apaga el pool de hilos de manera ordenada al terminar la ejecución de la recursión principal.

```
1     public static void main(String[] args) throws Exception {
2         Scanner sc = new Scanner(System.in);
3
4         System.out.print("Ingresa una URL: ");
5         String dir = sc.nextLine();
6
7         System.out.print("Ingresa el nivel de profundidad explicito (ej.
            3): ");
8         int nivelExplicito = sc.nextInt();
9
10        URI uri = new URI(dir);
11        String host = uri.getHost();
12    }
```

```

13     createDirectory(host);
14
15     descarga(uri.toString(), host, nivelExplicito);
16
17     poolHilos.shutdown();
18     sc.close();
19 }

```

Listing 2: Implementación del método main

2.2. Método descarga

Es el núcleo recursivo encargado de procesar las páginas HTML. Recibe como argumento un entero `nivelRestante`. Si la URL ya se encuentra en el conjunto `visited`, el método interrumpe su ejecución para evitar bucles. Configura una conexión síncrona mediante el método `GET` simulando un navegador comercial y deshabilita el reenvío automático de redirecciones (`setInstanceFollowRedirects(false)`) para atrapar y reportar manualmente estados como 301, 302, 403 y 404[1].

Si el estado es exitoso (200 OK), descarga el recurso a disco usando una tubería temporal de bytes de 8 KB. Posteriormente, si el tipo de contenido corresponde a texto HTML, utiliza Jsoup para extraer elementos de presentación (`img`, `link`, `script`) y los manda al pool de hilos de forma asíncrona[2]. Finalmente, si `nivelRestante > 0`, extrae las etiquetas `<a>` e invoca recursivamente a `descarga` decrementando explícitamente el nivel en una unidad (`nivelRestante - 1`).

```

1  private static void descarga(String urlStr, String baseDir, int
2      nivelRestante) {
3      if (visited.contains(urlStr)) return;
4      visited.add(urlStr);
5
6      try {
7          URI uri = new URI(urlStr);
8          URL url = uri.toURL();
9          HttpURLConnection connection = (HttpURLConnection) url.
10             openConnection();
11          connection.setRequestMethod("GET");
12          connection.setRequestProperty("User-Agent", "Mozilla/5.0");
13          connection.setInstanceFollowRedirects(false);
14
15          int responseCode = connection.getResponseCode();
16
17          if (responseCode == HttpURLConnection.HTTP_MOVED_PERM ||
18              responseCode == HttpURLConnection.HTTP_MOVED_TEMP) {
19              String nuevaUrl = connection.getHeaderField("Location");
20              System.out.println("Redireccion 301/302 -> " + nuevaUrl);
21              descarga(nuevaUrl, baseDir, nivelRestante);
22              return;
23          } else if (responseCode == HttpURLConnection.HTTP_FORBIDDEN) {
24              System.out.println("Error 403 (Prohibido) en: " + urlStr);
25              return;
26          } else if (responseCode == HttpURLConnection.HTTP_NOT_FOUND) {

```

```

24         System.out.println("Error 404 (No encontrado) en: " +
25             urlStr);
26         return;
27     } else if (responseCode != HttpURLConnection.HTTP_OK) {
28         System.out.println("Error " + responseCode + " en: " +
29             urlStr);
30         return;
31     }
32
33     String slashPath = uri.getPath();
34     if (slashPath == null || slashPath.equals("/") || slashPath.
35         isEmpty()) {
36         slashPath = "/index.html";
37     } else if (slashPath.endsWith("/")) {
38         slashPath += "index.html";
39     }
40
41     String filePath = "./" + baseDir + slashPath;
42     Path path = Paths.get(filePath);
43     Files.createDirectories(path.getParent());
44
45     ByteArrayOutputStream baos = new ByteArrayOutputStream();
46     try (InputStream in = connection.getInputStream()) {
47         byte[] buffer = new byte[8192];
48         int bytesRead;
49         while ((bytesRead = in.read(buffer)) != -1) {
50             baos.write(buffer, 0, bytesRead);
51         }
52         Files.write(path, baos.toByteArray());
53     }
54     System.out.println("Descargado [Nivel restante: " +
55         nivelRestante + "]: " + filePath);
56
57     String contentType = connection.getContentType();
58     if (contentType != null && contentType.contains("text/html"))
59     {
60         File file = new File(filePath);
61         Document doc = Jsoup.parse(file, "UTF-8", urlStr);
62
63         Elements incrustados = doc.select("img[src], link[rel=
64             stylesheet], script[src]");
65         for (Element elem : incrustados) {
66             String absUrl = elem.hasAttr("src") ? elem.absUrl("src
67                 ") : elem.absUrl("href");
68             if (!absUrl.isEmpty() && !visited.contains(absUrl)) {
69                 poolHilos.submit(() -> descargarElementoConHilo(
70                     absUrl, baseDir));
71             }
72         }
73
74         if (nivelRestante > 0) {
75             Elements hipervinculos = doc.select("a[href]");
76             for (Element link : hipervinculos) {
77                 String absUrl = link.absUrl("href");

```



```

70         if (absUrl.startsWith("http")) {
71             descarga(absUrl, baseDir, nivelRestante - 1);
72         }
73     }
74 }
75 }
76 } catch (Exception e) {
77     System.err.println("Error en " + urlStr + ": " + e.getMessage
78         ());
79 }

```

Listing 3: Implementación del método descarga con cuenta regresiva

2.3. Método descargarElementoConHilo

Es la tarea secundaria diseñada exclusivamente para procesar la descarga paralela de imágenes, estilos y scripts de forma asíncrona[1]. Al ejecutarse dentro de uno de los 5 hilos de trabajo del pool, abre un canal binario directo hacia el disco duro utilizando un buffer optimizado de 8 KB, garantizando que el hilo principal no sufra retrasos por la latencia del canal I/O.

```

1  private static void descargarElementoConHilo(String urlStr, String
2      baseDir) {
3      if (visited.contains(urlStr)) return;
4      visited.add(urlStr);
5
6      try {
7          URL url = new URI(urlStr).toURL();
8          HttpURLConnection connection = (HttpURLConnection) url.
9              openConnection();
10         connection.setRequestMethod("GET");
11         connection.setRequestProperty("User-Agent", "Mozilla/5.0");
12
13         if (connection.getResponseCode() == HttpURLConnection.HTTP_OK)
14             {
15                 String pathName = url.getPath();
16                 if (pathName == null || pathName.equals("/")) pathName = "
17                     /recurso_" + System.currentTimeMillis();
18
19                 Path targetPath = Paths.get("./" + baseDir + pathName);
20                 Files.createDirectories(targetPath.getParent());
21
22                 try (InputStream in = connection.getInputStream();
23                     OutputStream out = Files.newOutputStream(targetPath)) {
24                     byte[] buffer = new byte[8192];
25                     int bytesRead;
26                     while ((bytesRead = in.read(buffer)) != -1) {
27                         out.write(buffer, 0, bytesRead);
28                     }
29                 }
30                 System.out.println(" [Hilo " + Thread.currentThread().
31                     getName() + " finalizo] -> " + targetPath.getFileName())

```

```
26         }  
27     } catch (Exception e) {}  
28 }
```

Listing 4: Implementación del método de descarga concurrente para assets

3. Ejecución

Para verificar el comportamiento correcto del control de profundidad descendente y el pool de hilos, se ejecutó el programa en consola apuntando a un servidor web local de pruebas de la escuela.

3.1. Prueba con Profundidad 0 (Solo página base y sus elementos)

Al introducir un nivel de profundidad igual a 0, el programa descarga de forma exitosa el archivo `index.html`. Dado que la cuenta regresiva inicia en 0, la condición `if (nivelRestante > 0)` es falsa, impidiendo la descarga de páginas enlazadas, pero permitiendo que el pool de hilos procese en paralelo los elementos incrustados inmediatos de esa página.

```
Ingresa una URL: http://148.204.58.221/axel/aplicaciones/sockets/java/
Ingresa el nivel de profundidad : 0
Descargado [Nivel : 0]: ./148.204.58.221/axel/aplicaciones/sockets/java/index.html
[Hilo] -> blank.gif
[Hilo] -> compressed.gif
[Hilo] -> text.gif
[Hilo] -> folder.gif
[Hilo] -> back.gif
[Hilo] -> unknown.gif
[Hilo] -> layout.gif
```

3.2. Prueba con Profundidad 2 (Descarga acotada)

Al configurar la profundidad explícita en 2, el hilo principal realiza el descenso a través de los hipervínculos incrementando los niveles de anidamiento de directorios locales y mostrando en consola cómo se reduce el permiso de exploración en cada salto recursivo.

```
Ingresa una URL: http://148.204.58.221/axel/aplicaciones/sockets/java/
Ingresa el nivel de profundidad : 1
Descargado [Nivel : 0]: ./148.204.58.221/axel/aplicaciones/sockets/java/index.html
[Hilo] -> blank.gif
[Hilo] -> text.gif
[Hilo] -> folder.gif
[Hilo] -> back.gif
[Hilo] -> compressed.gif
[Hilo] -> unknown.gif
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/index.html
[Hilo] -> layout.gif
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/index.html
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/index.html
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/index.html
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Cliente_0.java
```

Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Cliente_ftp.java
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/DatagramPacket.t
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/DatagramaECO.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/DatagramaHM.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/DatagramaObjeto
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/EchoClient.java
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/EchoServer.java
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/EjJsonSimple.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/EjemploJACKSON.z
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/FTP.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/FlujoArchivo.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/FlujoArchivo_mod
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/FlujoEco.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/FlujoEco/index.h
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Flujo_Objeto.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Fx.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/GUI.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/HolaMundo.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/HolaMundo999.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Multicast.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/MulticastClient.
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/MulticastClient2
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/MulticastServer2
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Mutex.java
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Objeto.java
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Othello.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Othello2.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/PacketReceiveDem
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/PacketSendDemo.j
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Server2.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Server_tmp.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/ServidorWeb3.jav
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Servidor_0.java
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Servidor_ftp.jav
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/Tarjeta.java
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/TiposDatos.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/canales/index.ht
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/captura_HTTP.pdf
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/clase/index.html
 [Hilo] -> image2.gif
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/datagrama.txt
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/hilos/index.html
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/mp3.zip
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/multicast/index.
 Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/multicastC-JAVA.

Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/recurso/index.ht
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/serializacion/in
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/servidor/index.h
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/tmp1/index.html
Descargado [Nivel : 1]: ./148.204.58.221/axel/aplicaciones/sockets/java/udp/index.html

4. Conclusiones

Esta práctica permite entender el formato de una solicitud y respuesta de tipo GET, además sobre como se puede manipular su payload. El uso de hilos para descargas concurrentes permite hacer una descarga más eficaz del payload. Esto permite entender lo que se conoce como Web Crawling. Los elementos del payload constantemente tienen elementos incrustados los cuales tambien se deben atender, para lograrlo se usó una biblioteca de parseo para identificar dichos elementos incrustados y descargarlos mediante un hilo del pool de 5 hilos implementados.

5. Referencias

Referencias

- [1] Oracle, “Class HttpURLConnection,” Java Standard Edition, 2025. [En línea]. Disponible: <https://docs.oracle.com/en/java/javase/>. [Accedido: 24-jun-2026].
- [2] Jsoup, “jsoup: Java HTML Parser.” [En línea]. Disponible: <https://jsoup.org/>. [Accedido: 24-jun-2026].
- [3] GeeksforGeeks, “What is a Web Crawler and where is it used?,” 2024. [En línea]. Disponible: <https://www.geeksforgeeks.org/>. [Accedido: 24-jun-2026].
- [4] Mozilla Developer Network, “An overview of HTTP.” [En línea]. Disponible: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>. [Accedido: 24-jun-2026].